Chapter 5 — Introduction to data cleaning

After collection and annotation, raw tables still need preparation before reliable analysis or supervised training

Learning objectives

- Distinguish cleaning from preprocessing, name four practical motives for both phases
- Recognize schema discipline when joining multiple corpora

Why cleaning matters

- Without cleaning, missing fields, duplicates, inconsistent encodings
- Cross-corpus work makes those defects visible at pipeline scale

Example 5.1 — Missing target values in classification

- Example 5.1 — hands-on module
- Example 5.1 shows a fraud table with blank labels
- Supervised training either drops those rows or invents labels
- Explore the chapter example module
- View files: `modules/chapter5/example1/`

Example 5.2 — Dropping unused identifier columns

- Example 5.2 — hands-on module
- Example 5.2 removes high-cardinality identifiers such as raw transaction IDs that never
- Memory and training time improve without changing the predictive schema
- Explore the chapter example module
- View files: `modules/chapter5/example2/`

Cleaning versus preprocessing

- This book treats cleaning and preprocessing as consecutive phases
- Non-model fields
- Preprocessing transforms cleaned values into representations algorithms expect

Example 5.3 — Duplicate retail transactions

- Example 5.3 — hands-on module
- Example 5.3 shows the same order ID appearing twice after a system retry
- Counting both rows inflates revenue and customer frequency until duplicates are removed or
- Cleaning precedes the scaling and encoding examples introduced later in the chapter
- View files: `modules/chapter5/example3/`

Four recurring motives

- Four motives recur
- Detailed failure modes appear in the next parts; remedies follow in Sections 5.3 and 5.4

Takeaways

- Cleaning makes values trustworthy; preprocessing makes them algorithm-ready
- Both follow annotation and precede exploration-heavy modeling work
- Schema alignment across sources is itself a cleaning task

Next

- Complete the quiz for this part
- Continue to missing data, MCAR, MAR, MNAR mechanisms and their causes

Learning objectives

- Define MCAR, MAR
- MNAR, match each to a short scenario
- List typical causes of missing fields in real pipelines

MCAR — missing completely at random

- The probability a value is missing does not depend on observed or unobserved data
- Example 5.8 sketches a system error that omits records uniformly
- Simple deletion or mean imputation may be less biased when MCAR truly holds

MAR — missing at random

- Under MAR, missingness depends on observed variables but not on the missing value itself
- Example 5.9 shows income nonresponse related to age or employment status that is recorded
- MAR is the sweet spot for many principled imputation methods

MNAR — missing not at random

- Under MNAR, missingness depends on the unobserved value itself
- Example 5.10 shows high earners declining to report income
- MNAR demands explicit modeling, sensitivity analysis, or collection redesign

Example 5.8 — MCAR system error

- Example 5.8 — hands-on module
- The uniform omission pattern
- Explore the chapter example module
- View files: `modules/chapter5/example8/`

Causes of missing data

- Disk or sensor failures truncating files
- Example 5.11 shows skipped sensitive items; Example 5.13 shows truncated sensor logs
- The cause hints at the mechanism

Why mechanism matters

- Listwise deletion is simple but wasteful under MAR if informative rows are dropped
- Mean imputation understates variance
- MNAR fixes require domain judgment
- Chapter 6 exploration helps detect patterns

Takeaways

- Classify missingness before imputing
- MNAR are not interchangeable labels, they dictate which repairs are defensible
- Causes range from human behavior to infrastructure failure

Next

- Complete the quiz for this part
- Continue to duplicates and inconsistent encodings

Learning objectives

- Explain duplicate impact on aggregates and training
- Describe how free-text categories hide structure

Duplicate data — causes and impact

- Duplicates arise from system retries, merged exports, copy-paste ETL
- Example 5.14 shows duplicate purchase rows
- Deduplication is not cosmetic, it changes business numbers

Example 5.14 — Duplicate customer purchase rows

- Example 5.14 — hands-on module
- The retail retry pattern
- Explore the chapter example module
- View files: `modules/chapter5/example14/`

Inconsistent data — formats and units

- Inconsistent data is common when sources span departments or time periods
- Example 5.18 mixes date formats, gender encodings, and measurement units in one table
- Example 5.19 shows customer ID formats that block joins
- Example 5.20 splits Male versus M into separate categories
- Example 5.21 shows weeks lost normalizing free-text cities

Example 5.18 — Mixed dates, gender, and units

- Example 5.18 — hands-on module
- A checklist of silent parse failures
- Explore the chapter example module
- View files: `modules/chapter5/example18/`

Join and aggregation failures

- Inconsistent keys prevent reliable joins across tables
- Mixed currencies or units make sums meaningless until converted
- Split labels inflate cardinality and starve rare categories of support
- Exploration in Chapter 6 surfaces many of these patterns

Takeaways

- Duplicates change revenue, sample weights, and runtime
- Inconsistency breaks time logic and joins
- Standardize formats early with explicit parsing rules and canonical category maps

Next

- Complete the quiz for this part
- Continue to outliers, irrelevant features, and class imbalance

Learning objectives

- Distinguish erroneous outliers from legitimate extremes
- Describe why rare classes break accuracy metrics

Outliers — definition and causes

- Outliers deviate sharply from the bulk of a distribution
- Causes include typos, sensor faults, and genuine rare events
- Example 5.22 shows an extra zero in blood pressure
- Treatment depends on which case you face

Handling outliers

- Replacing spikes with robust statistics such as the median
- Examples 5.25 through 5.28 illustrate drop, log, cap, and median-replace patterns
- Blind removal of all extremes can delete signal

Example 5.22 — Extra zero in blood pressure

- Example 5.22 — hands-on module
- A classic entry error
- Explore the chapter example module
- View files: `modules/chapter5/example22/`

Irrelevant features

- Irrelevant columns add noise without predictive value
- Example 5.29 lists color or pet features unrelated to a loan default target
- Example 5.30 notes harder interpretation
- Correlation and domain review help prune fields

Example 5.29 — Irrelevant color and pet features

- Example 5.29 — hands-on module
- Example 5.29 motivates feature selection before modeling
- Explore the chapter example module
- View files: `modules/chapter5/example29/`

Imbalanced data

- Imbalance means one class dominates, Example 5.33 shows fraud at one percent of events
- Majority-class classifiers can report high accuracy while missing almost every fraud case
- Responses include oversampling, class weights

Takeaways

- Outlier handling requires domain judgment
- Irrelevant features waste compute and invite overfitting
- Imbalance makes accuracy misleading, plan metrics and resampling before training

Next

- Complete the quiz for this part
- Continue to cleaning techniques, imputation, deduplication, standardization

Learning objectives

- Compare listwise deletion with available-case analysis, choose mean, median
- Hot-deck imputation, remove duplicates in pandas-style workflows
- Flag outliers with IQR rules

Handling missing data — deletion

- Listwise deletion drops any row with a missing value in selected columns
- Available-case analysis keeps rows for each analysis using the fields present
- Choose based on mechanism and how much data you can afford to lose

Imputation methods

- Mean imputation, Example 5.39 for age, preserves the sample mean but shrinks variance
- Median imputation, Example 5.41 for skewed square footage, is robust to tails
- Hot-deck imputation
- Document imputation inside train splits to avoid leakage

Example 5.39 — Mean impute missing age

- Example 5.39 — hands-on module
- The simple numeric fill
- Explore the chapter example module
- View files: `modules/chapter5/example39/`

Removing duplicates

- Near-duplicate customer names and repeated transaction keys should be collapsed with
- Example 5.42 shows fuzzy duplicate risk
- Use stable keys when available

Standardizing formats

- Example 5.43 normalizes currency symbols across markets before modeling
- Date parsing, lowercasing product names
- Record parsing rules in preprocessing metadata

Outliers and feature selection

- Example 5.44 flags extreme income with the interquartile range
- Winsorization caps tails instead of deleting rows
- Example 5.45 drops one of two highly correlated bathroom features to reduce redundancy
- Pair outlier rules with domain review

Takeaways

- Match the repair to the defect and mechanism
- Deletion is simple but costly; imputation carries bias risk
- Standardization enables joins; IQR and correlation pruning stabilize training

Next

- Complete the quiz for this part
- Continue to preprocessing, scaling and encoding cleaned columns

Learning objectives

- Choose between min-max scaling and standardization
- State leakage risks for target encoding

Normalization and scaling

- Min-max normalization maps values to a bounded range such as zero to one
- Standardization subtracts the mean and divides by standard deviation
- Tree models often need less scaling but still benefit from consistent units

Example 5.46 — Scale age and income for KNN

- Example 5.46 — hands-on module
- Example 5.46 revisits the scale mismatch motif from Section 5.1
- Explore the chapter example module
- View files: `modules/chapter5/example46/`

One-hot encoding

- One-hot encoding creates a binary column per category level
- Use for nominal categories without intrinsic order
- High-cardinality fields may need hashing, embedding, or target encoding instead

Label encoding

- Label encoding assigns integers to categories
- Do not use arbitrary integer codes for nominal data with unordered labels

Target encoding

- Target encoding replaces a category with the mean outcome for that category
- Powerful but leak-prone
- Never encode using the full dataset before splitting

Takeaways

- Scale when magnitude drives the algorithm
- Match encoding to cardinality and ordinality
- Guard target statistics against leakage

Next

- Complete the quiz for this part
- Continue to binning, feature engineering, and transforms

Learning objectives

- Explain equal-width versus frequency binning, construct ratio and interaction features
- Choose log or polynomial expansions when skew or nonlinearity demands them

Binning numerical data

- Binning groups continuous values into intervals
- Example 5.51 uses equal-width age bins
- Binning reduces noise and handles nonlinear effects but loses granularity

Adding new features

- Example 5.53 adds a squared term
- Domain ratios often outperform raw stacks of columns
- Avoid generating features that duplicate identifiers or leak targets

Example 5.55 — Engineer price per square foot

- Example 5.55 — hands-on module
- The classic ratio feature
- Explore the chapter example module
- View files: `modules/chapter5/example55/`

Dimensionality reduction and selection

- High-dimensional sparse tables benefit from principal components
- Selection reduces overfitting and training time
- Keep interpretability requirements in mind for regulated domains

Log and polynomial transforms

- Log transforms compress right tails, Example 5.26 on income
- Polynomial expansions, Example 5.56, let linear models capture curvature
- Apply transforms before splitting or fit parameters on training folds only
- Inverse transforms may be needed for interpretation

Takeaways

- Engineering encodes domain knowledge
- Binning and transforms trade resolution for stability
- Document every derived column for reproducibility and explainability

Next

- Complete the quiz for this part
- Continue to Python tools that implement these steps at scale

Learning objectives

- Perform dropna, fillna
- Drop_duplicates in pandas, apply StandardScaler and OneHotEncoder from scikit-learn
- Outline an end-to-end cleaning walkthrough

pandas for cleaning

- Pandas is the default toolkit for tabular cleaning
- Example 5.57 shows dropna and drop_duplicates in one script
- Example 5.59 walks an end-to-end cleaning template from raw extract to model-ready frame

Example 5.57 — Pandas dropna and drop duplicates

- Example 5.57 — hands-on module
- The minimal cleaning API surface
- Explore the chapter example module
- View files: `modules/chapter5/example57/`

Example 5.57 — listing

```
import pandas as pd
df = pd.read_csv("data.csv")
df = df.dropna()  # Removing rows with missing values
df = df.drop_duplicates()  # Removing duplicate rows
```

scikit-learn for preprocessing

- Scikit-learn provides fit-transform estimators
- Example 5.60 standardizes age and salary; Example 5.61 one-hot encodes gender
- Fit on training data only to prevent leakage

Example 5.59 — Pandas end-to-end cleaning walkthrough

- Example 5.59 — hands-on module
- Example 5.59 ties issues to fixes in one narrative
- Explore the chapter example module
- View files: `modules/chapter5/example59/`

Example 5.59 — listing

```
import pandas as pd

# Load data
df = pd.read_csv("customers.csv")

# 1. Handling Missing Data
df['age'] = df['age'].fillna(df['age'].mean())
df = df.dropna(subset=['name', 'email'])

# 2. Removing Duplicates
df = df.drop_duplicates(subset=['email'])

# 3. Standardizing Column Formats
df['name'] = df['name'].str.title()

# 4. Encoding Categorical Data
# ...
```

When to use which layer

- Use pandas for exploratory fixes and bespoke string parsing
- Use sklearn transformers when the same preprocessing must repeat across folds and
- Keep business rules and unit conversions visible in notebooks or pipeline comments

Takeaways

- Pandas repairs tables; sklearn standardizes repeatable transforms
- Fit transformers on training splits
- Example modules provide runnable templates for this chapter

Next

- Complete the quiz for this part
- Continue to pipelines, automation, visualization, and a brief R contrast

Learning objectives

- Explain why pipelines prevent leakage
- Contrast dplyr/tidyr with pandas

R: dplyr and tidyr

- In R workflows
- Example 5.58 shows filter and select patterns analogous to pandas
- Teams standardized on Python may still encounter R extracts

sklearn pipelines

- Pipelines chain imputers, scalers, encoders
- This prevents statistics computed on validation data from leaking into features
- Serialize pipelines with the model for consistent deployment preprocessing

Automation: Featuretools and AutoML

- Featuretools performs deep feature synthesis on relational tables
- AutoML platforms wrap search over imputers, encoders, and model families
- Automation accelerates baselines but still requires domain review to drop leaky or

Example 5.74 — Deep feature synthesis on transactions

- Example 5.74 — hands-on module
- Example 5.74 shows automated aggregates across related tables
- Explore the chapter example module
- View files: `modules/chapter5/example74/`

Visualization for cleaning

- Box plots expose outlier tails
- Visual checks complement rule-based flags from earlier parts and align with exploratory

Takeaways

- Pipelines encode reproducible preprocessing
- Automation proposes features; humans curate them
- Visual diagnostics catch defects rules miss

Next

- Complete the quiz for this part
- Continue to challenges, best practices, and governance habits

Learning objectives

- Explain why accuracy misleads on imbalanced data, describe currency and unit mixing risks
- List documentation, validation
- Version-control habits

Recurring challenges

- Missing data mechanisms complicate imputation choices
- Inconsistent cross-source formats consume analyst time
- Outliers mix errors with rare truth
- Imbalance steers models toward majority classes
- Duplicates inflate counts
- Transformations can leak target information if fit globally

Example 5.62 — Rare fraud class distorts accuracy

- Example 5.62 — hands-on module
- Example 5.62 shows fraud at a tiny fraction of events
- Explore the chapter example module
- View files: `modules/chapter5/example62/`

Example 5.63 — Convert currencies before modeling

- Example 5.63 — hands-on module
- Example 5.63 reminds teams to harmonize currencies and units before comparing amounts
- Mixed units silently rank features by exchange noise instead of behavior
- Standardize early and record conversion tables
- View files: `modules/chapter5/example63/`

Best practices overview

- Handle missing data with explicit mechanism assumptions
- Profile data before and after each step
- Document parsing rules, imputation choices, and outlier policies
- Separate train, validation, and test before fitting transformers
- Use version control for datasets and preprocessing code
- Review features for leakage and fairness implications connected to Chapter 7

Validation and reproducibility

- Fit scalers, encoders, and imputers inside training folds
- Store random seeds and library versions
- Write preprocessing metadata alongside exported tables so another team can replay the

Takeaways

- Challenges are expected, plan time for them
- Metrics must match the defect, especially under imbalance
- Documentation and leakage control are part of cleaning, not optional extras

Next

- Complete the quiz for this part
- Continue to retail and healthcare case studies plus hands-on module activities

Learning objectives

- Outline cleaning steps for an e-commerce segmentation table and a clinical outcomes table
- Locate module labs for issue spotting and pandas repair templates

Retail: customer segmentation

- An e-commerce firm segments customers by purchase behavior
- Challenges include missing age and frequency fields, inconsistent product capitalization
- Solutions: median or mode imputation, currency harmonization, text normalization
- Cleaned data supports clustering and targeted campaigns

Healthcare: predicting disease outcomes

- Clinical records combine age, history, labs, and outcomes
- Missing labs, outlier vitals, and inconsistent smoker labels are common
- Solutions: principled imputation, categorical standardization, IQR outlier review
- Preprocessing improves allocative and diagnostic workflows when documented

Example 5.64 — Retail purchase record schema

- Example 5.64 — hands-on module
- Example 5.64 introduces a practice table with missing prices and duplicate transactions
- Explore the chapter example module
- View files: `modules/chapter5/example64/`

Example 5.65 — Guided pandas cleaning template

- Example 5.65 — hands-on module
- Example 5.65 provides a guided repair script
- Explore the chapter example module
- View files: `modules/chapter5/example65/`

Example 5.65 — listing

```
import pandas as pd

df = pd.read_csv("data.csv")

df['price'] = df['price'].fillna(df['price'].mean())
df.dropna(subset=['customer_id'], inplace=True)

df.drop_duplicates(inplace=True)

df['product'] = df['product'].str.lower()
df['purchase_date'] = pd.to_datetime(df['purchase_date'])

z_scores = (df['price'] - df['price'].mean()) / df['price'].std()
df = df[z_scores < 3]

df.to_csv("cleaned_data.csv", index=False)
```

From case study to production

- Both cases share a pattern
- Retail emphasizes revenue integrity
- Fairness review may follow in Chapter 7

Takeaways

- Domain context chooses imputation and outlier rules
- Case studies connect Sections 5.2–5.5 into narratives
- Module activities supply hands-on repetition without requiring live execution on the

Next

- Complete the quiz for this part
- Continue to advanced topics, Bayesian imputation, streaming data

Learning objectives

- Contrast Bayesian and deep imputation, describe sliding-window streaming preprocessing
- Explain how documentation supports explainable AI

Advanced imputation

- Bayesian imputation models missing values with uncertainty
- Autoencoder imputation, Example 5.71, learns nonlinear reconstructions for retail baskets
- Both beat naive means when patterns are complex but need more data and compute

Example 5.70 — Bayesian imputation with clinical priors

- Example 5.70 — hands-on module
- The uncertainty-aware clinical case
- Explore the chapter example module
- View files: `modules/chapter5/example70/`

Real-time and streaming preprocessing

- Streaming finance, IoT, and vitals demand low-latency transforms
- Challenges include latency, arriving corruption, and scale
- Sliding windows, Example 5.72 on prices
- Batch assumptions fail when data never stops

Example 5.72 — Rolling window for streaming prices

- Example 5.72 — hands-on module
- Example 5.72 maintains a rolling feature window for aggregates
- Explore the chapter example module
- View files: `modules/chapter5/example72/`

Automated pipelines and explainability

- Automated pipelines orchestrate cleaning stages across storage systems
- For explainable AI

Example 5.75 — Document imputation for auditable models

- Example 5.75 — hands-on module
- Example 5.75 ties preprocessing logs to model cards
- Explore the chapter example module
- View files: `modules/chapter5/example75/`

Looking ahead

- Chapter 6 continues with exploration that detects many defects this chapter repairs
- Chapter 7 connects preprocessing to fairness
- Strong cleaning documentation makes both downstream chapters more trustworthy

Takeaways

- Advanced methods extend, not replace, core taxonomy and tools
- Streaming and explainability impose new constraints on familiar transforms
- Document every stage for production and audit

Next

- Complete the quiz for this part
- Pause for the quiz